

A Tiny Vision-Language Model that Spells What It Sees

RLS Entrance Challenge | Youssef | code: github.com/youssefkobbi/RLS-Entrance-challenge26 | final commit: [\[TBD: git hash\]](#)

1. Problem

The task is to build the smallest honest version of a vision-language model. The input is a 64x64 image with one or two coloured shapes (ShapeScenes, generated by the RLS script with seed 42). The output is one word with no spaces that describes the scene, such as *largeredcircle* or *smallbluesquareleftoflargeyellowtriangle*, written one letter at a time from a 27-token vocabulary (26 letters and an end token). Because the right answer is known exactly, I can measure what the model really learned. I wanted to learn three things: how an image becomes tokens that a text decoder can read, whether the model really uses the image (the blind baseline), and how the training step behaves on a CPU and on a GPU.

2. Architecture

letters so far (B,T) -> embedding (B,T,d_model) joins the sequence at step 4; greedy decoding repeats steps 4-6 until <eos>

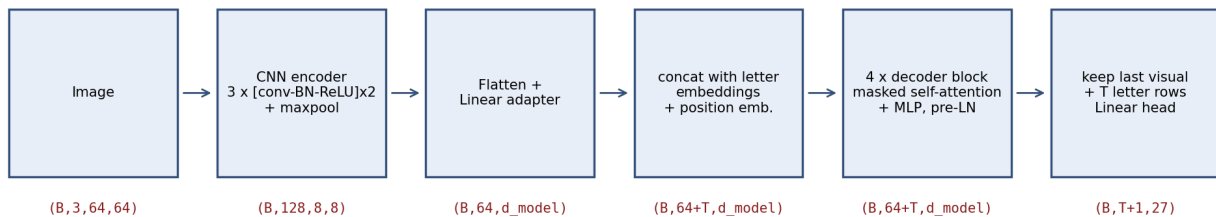
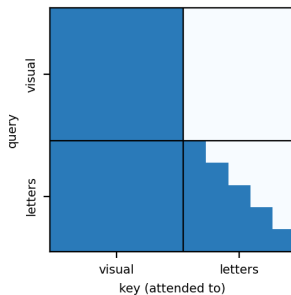


Figure 1. The pipeline with tensor shapes. B is the batch size, T the number of letters fed to the decoder, d_model = 128. The model has 1,118,203 parameters.



Design choices. (a) **CNN encoder:** three stages of two 3x3 convolutions with batch norm and ReLU, each followed by a 2x2 max-pool, so the image goes 64 -> 32 -> 16 -> 8 pixels wide. The 8x8 grid of 128-channel features gives 64 visual tokens: few enough to keep the sequence short (64 + up to 45 = 109 tokens) but still a grid, so left/right and above/below stay recoverable. Pooling throws away the exact position inside each 2x2 window, so I rely on the learned position embeddings to keep the coarse layout. (b) **Adapter:** one linear layer per grid cell, so a visual token has the same size as a letter embedding and the decoder cannot tell them apart. (c) **Mask (prefix-LM, Figure 2):** visual tokens attend to each other, because the whole image is given at once and there is no future to hide; letters are causal and can see every visual token. (d) **Which output predicts what:** the logit of the last visual token predicts letter 1, the logit of letter i predicts letter $i+1$, and the last letter predicts <eos>, so a word of L letters gives L+1 targets and no start token is needed.

Figure 2. Attention mask, dark = may attend.

3. Implementation

Attention (src/model/attention.py) is written with tensor operations only: one linear layer makes Q, K and V, the heads are contiguous channel slices, the scores are $QK^T / \sqrt{d_k}$, forbidden pairs are filled with the most negative float before a softmax over the keys, and the heads are merged and passed through an output projection. Dividing by $\sqrt{d_k}$ keeps the score variance near 1, so the softmax does not saturate. **Decoder block:** pre-LayerNorm, $x = x + \text{Attn}(\text{LN}(x))$, then $x = x + \text{MLP}(\text{LN}(x))$, with a GELU MLP of width 512 and dropout 0.1. **Loss masking:** words have different lengths, so targets are padded with -100 and cross-entropy uses ignore_index; padded inputs use letter id 0, which is safe because the causal mask stops real positions from seeing them. **Generation:** greedy, one letter per step until <eos> or 45 letters; the CNN runs once and only the decoder is re-run each step (no KV cache). **Evaluation:** exact match, plus a parser that cuts a possibly misspelt word into size / colour / shape [/ relation / size / colour / shape] by dynamic programming over edit distance to the vocabulary, so *largeredcirle* still counts as *large, red, circle*.

How I checked correctness. The provided test compares my attention to F.scaled_dot_product_attention with five masks (none, causal in 2-D and 4-D form, random per sample, prefix-LM) at 1e-5 in float32, and further tests check that a masked key has no influence, that causal outputs do not depend on later letters, and that every parameter gets a gradient. The provided shape tests check the whole model, and my own tests cover the tokenizer, padding and parser. Finally E0 (below) checks that the pipeline can learn at all.

4. Training setup

Optimizer	AdamW, lr 1e-3, weight decay 0.01, gradient clipping 1.0	Schedule	200 warm-up steps, then cosine to 5% of lr
-----------	--	----------	--

Batch / epochs	128 images, 30 epochs (157 steps per epoch)	Regularisation	dropout 0.1
Data	train 20,000, val 2,000, test 2,000 (heldout 2,000, not analysed)	Seeds	0, 1, 2 (dataset seed 42)
Checkpoint	best validation loss	Hardware	Colab Tesla T4 for training and S1; laptop CPU for E0 and S1

5. Experiments

E0, overfit one batch. Question: can the pipeline learn at all? 32 fixed training images, dropout off, 400 steps of AdamW at lr 1e-3. Success means the loss goes to almost zero and greedy decoding reproduces every word. **E1, blind baseline.** Question: does the model use the image? *Hypothesis, written before running:* with all-zero images the model only knows the grammar of the words, so attribute accuracy should be at chance (size about 0.5, colour, shape and relation about 0.25), exact match close to 0, while teacher-forced letter accuracy stays high because the letters inside a part are pure spelling. Baseline: the same model with the real image. One variable changed: images replaced by zeros in training and evaluation. Fixed: architecture, parameter count, optimizer, learning rate, batch size, epochs, seeds and splits. **S1, throughput.** Time of a full training step (forward, loss, backward, optimizer) on a batch already in device memory, for batch sizes 1, 16, 64 and 256. Traps handled: 5 warm-up steps are discarded (first calls pay one-off costs such as CUDA context, cuDNN algorithm choice and allocator growth); torch.cuda.synchronize() is called before the timer starts and before it is read, because GPU calls return before the work is done; data loading is **outside** the timer. 5 repeats of 10 steps each, mean and standard deviation reported.

6. Results

Metric (test split)	Main model	Blind E1
Exact match	[TBD: run]	[TBD: run]
Size / colour / shape / relation acc.	[TBD: run]	[TBD: run]
Letter acc. (all / first / inside)	[TBD: run]	[TBD: run]

Table 1. Test-split accuracy, mean +- std over seeds (std is the sample standard deviation). Confgs: configs/baseline.yaml and configs/blind.yaml.

[TBD: loss_curves.png of a baseline run]

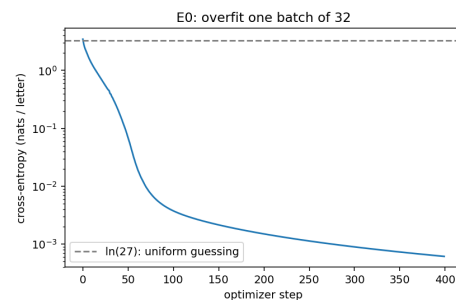


Figure 3 (left). Train and validation loss of the main model (log scale) [TBD: caption: what to notice]. **Figure 4 (right).** E0: E0 final loss 0.0006 (from 3.48), greedy exact match 1.00 on the memorised batch, run on a laptop CPU. The loss falls far below $\ln(27) = 3.30$, the value of uniform guessing.

Batch size	T4 GPU (Colab)	CPU, Colab (1 thread)	CPU, laptop (8 threads)	GPU peak mem (MiB)
1	83 +- 2	23 +- 1	22 +- 2	43
16	1163 +- 82	33 +- 3	64 +- 6	184
64	1469 +- 4	29 +- 1	68 +- 2	651
256	1465 +- 4	28 +- 0	62 +- 4	2510

Table 2 (S1). Training throughput in images per second (mean +- std of 5 repeats), data loading excluded. Hardware: Colab Tesla T4 with Xeon 2.0 GHz (torch 2.11, CUDA 12.8); laptop AMD CPU (torch 2.4.1). The two CPU columns differ in thread count and torch version, so they are not a like-for-like comparison.

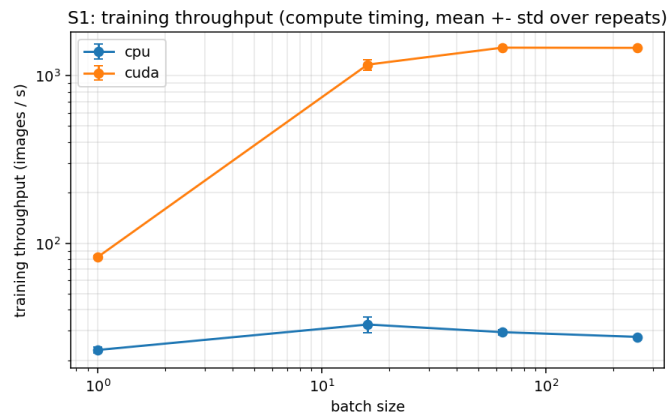


Figure 5. S1 on the Colab machine (log-log). The GPU is ahead at every batch size and flattens above batch 64; the CPU curve is nearly flat.

7. Analysis

Where does the GPU stop being faster than the CPU? In the range I measured it never stops: even at batch 1 the T4 trains 83 images/s against 23 (Colab CPU) and 22 (laptop CPU), about 3.8x faster. Going from batch 1 to 16 raises the GPU throughput 14x for 16x more images, so at batch 1 the step is dominated by fixed per-step cost (launching many small kernels and Python overhead), not arithmetic. Above batch 64 the throughput is flat (~1465 images/s), so the GPU is then limited by compute; my rough estimate is about 0.9 GFLOP per image for a training step, i.e. roughly 1.4 TFLOP/s, well below the T4's fp32 peak. The CPU curves are almost flat from batch 16 on: the cores are already busy at small batches, so a larger batch adds no throughput. Peak GPU memory grows roughly linearly with batch size (43 MiB at batch 1, 2.5 GiB at 256), consistent with the saved activations dominating over the 1.1M parameters. A GPU can lose to a CPU when the work per step is tiny; this model at batch 1 is still too large for that to happen. I did not test smaller inputs or models, and I did not time data loading.

[TBD: E1 analysis after the runs: compare attribute accuracy of the main and blind model, explain the gap between letter accuracy and attribute accuracy, say whether the hypothesis was right, and what the evidence does not show.]

8. Limitations

I did only Level 1. I did not run my own experiment E3, a profiler breakdown of the training step, a memory estimate to compare with the measured peaks, or an error analysis on test-heldout, so I cannot say how well the model generalises to unseen colour-shape pairs. Only one architecture and one set of hyperparameters were trained, and three seeds give only a rough idea of the spread. The S1 comparison mixes different machines, thread counts and torch versions, and excludes data loading. The absolute-position embeddings and the max-pool may limit how well spatial relations are learned. With more time I would run the held-out evaluation, an experiment on the number of visual tokens, and a profiler trace to see whether the GPU is waiting on the Python loop.

9. References

Vaswani et al., *Attention Is All You Need*, NeurIPS 2017 (Section 3). Liu et al., *Visual Instruction Tuning* (LLaVA), 2023. A. Karpathy, *Let's build GPT: from scratch, in code* (video) and *A Recipe for Training Neural Networks* (blog). H. He, *Making Deep Learning Go Brrrr From First Principles* (blog). PyTorch documentation, *CUDA semantics*. AI assistance: see AI_USAGE.md in the repository.